4-1 · MinIO 오브젝트 스토리지 구축 · 정리·회고 0.5H

정리·회고

mc 명령을 한 장으로, 4-2 를 위한 상태 점검

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① mc 명령을 「하려는 일」로 찾을 수 있다

② 4-2 준비 상태(컨테이너 2·버킷 2·첫 파일)를 명령으로 확인한다

③ 자주 겪는 오류 네 가지의 원인과 해결을 안다

④ 4-2 Iceberg 에서 무엇을 만드는지 미리 안다

컨테이너는 켜 둔 채로 두거나 docker compose stop 만 — down -v 는 볼륨까지 지웁니다

오늘의 요약

mc 명령 요약 — 하려는 일로 찾기

하려는 일	명령
서버 등록	mc alias set 이름 URL 키 비밀키
버킷 만들기	mc mb local/버킷
목록	mc ls [--recursive] local/버킷
올리기 · 내려받기	mc cp 원본 대상 (방향으로 결정)
내용 보기	mc cat local/버킷/키
지우기	mc rm local/버킷/키
폴더 통째 동기화	mc mirror 로컬폴더 local/버킷
버저닝 · 용량	mc version enable · mc du

모든 명령 앞에 docker compose exec mc — 별칭 local 은 compose 의 MC_HOST_local 이 정한 것

다음 차시 준비

4-2 준비 상태 — down → up 뒤에도 그대로

상태	docker compose ps → 컨테이너 2개 Up
버킷	docker compose exec mc mc ls local → raw · warehouse
오브젝트	mc ls --recursive local/raw → 다섯 개
볼륨	docker compose down → up -d 뒤에도 같은 다섯 개
멈출 때	docker compose stop — down -v 는 볼륨까지 지운다

다섯 = hello.txt · demo/ 둘 · from-python.txt · orders/orders_2026-08-01.csv — cd ~/lakehouse 에서

오늘의 체크리스트

여덟 개를 스스로 확인해 보세요

1. 블록·파일·오브젝트 차이를 한 문장씩 말할 수 있다
2. 키의 슬래시가 이름의 일부임을 raw/demo/dt=... 로 확인했다
3. docker compose ps 에서 minio·mc 가 Up 이다
4. 컨테이너 안에서 minio:9000 은 되고 localhost:9000 은 안 된다
5. alias(local) 확인 → mb → ls 로 raw·warehouse 버킷을 만들었다
6. down → up 뒤에도 버킷·파일이 남는 것(볼륨)을 확인했다
7. boto3 로 다루고 endpoint_url 만 바꾸면 AWS 에서도 도는 것을 안다
8. 랜딩 존 raw/orders/ 엔 첫 배치만 — 연습 키는 raw/demo/

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
port is already allocated	9000·9001 을 다른 프로세스가 사용	docker ps 로 찾아 중지 또는 19000:9000 매핑
웹 콘솔 로그인 거부	환경변수 바꾼 뒤 재생성 안 함 · 옛 볼륨	down → up -d · 안 되면 down -v (주의)
mc alias set connection refused	mc 안에서 localhost:9000 사용	서비스 이름 http://minio:9000
boto3 SSL 오류	endpoint_url 을 https:// 로 적음	http://localhost:9000 운영은 반대로 https

안 될 때 순서는 언제나 같다 — docker compose ps → logs 서비스명 → 포트 충돌 확인

예상 질문

자주 나오는 질문 네 가지

1. **파일 가운데 한 줄만 고치려면?**
   안 된다 — 통째로 다시 올린다. 이 제약을 우회하는 것이 4-2 Iceberg 의 핵심 설계

2. **MinIO 를 실제 운영에 써도 되나요?**
   쓴다 — 여러 대로 묶어 이중화하고 인증을 제대로. 오늘은 한 대 구성이라 학습용

3. **AWS S3 와 완전히 같은가요?**
   API 는 거의 같고 일부 고급 기능(수명 주기 세부 옵션)은 다르다 — 기본 조작은 그대로

4. **버킷을 왜 raw 와 warehouse 로 나눴나요?**
   원본과 가공 결과를 섞지 않기 위해 — 원본은 절대 바꾸지 않는다는 3-5 의 원칙

다음 회차 예고

4-2 · Apache Iceberg 레이크하우스 구축

01
이론 1H
레이크하우스와
테이블 포맷

02
실습 1H
Spark + Iceberg
환경 구성

03
실습 1.5H
테이블 생성과
타임 트래블

04
정리 0.5H
정리·회고

주말 과제: 이미지 둘 미리 받기 — spark-iceberg 6.46GB · iceberg-rest 746MB(실측 24분) · ~/lakehouse 그대로

4-1 · 정리·회고 0.5H 마무리

다음 회차는 4-2 Apache Iceberg

MinIO 위에 테이블 포맷을 얹어 demo.lakehouse.orders 테이블을 만들고 과거 시점을 조회합니다

- ✓ 데이터레이크 한계와 테이블 포맷의 해법
- ✓ REST 카탈로그 + Spark + MinIO Compose
- ✓ VERSION AS OF 로 과거 시점 조회
- ✓ ALTER TABLE 로 스키마 안전하게 변경

준비할 것

주말에 이미지 둘(7GB) 받아 두기